
메카트로닉스

Lab01 DC모터 결과보고서

21101263 김범준

2026. 4. 15.

|| 목 차 ||

I . 실습 1 1

II . 실습 2 3

III . 실습 3 5

IV . 실습 4 8

Lab 1. DC 모터 PID 제어

I 실습 1. PWM을 이용한 DC 모터 구동

□ 실험 과정 및 결과

○ PWM 제어 로직 구현 및 레지스터 설정

- FPGA 내장 PWM 발생 회로의 레지스터(*DC_DRV_EN, *DC_PWM_R)를 제어하여 모터 드라이버를 활성화하고 Duty 비를 설정하였다.
- PWMOut 함수를 구현하여 인자값 -50 ~ 50 범위를 받아 실제 0 ~ 100% Duty로 변환하는 함수를 작성하였다. (변환 수식: $Register\ Value = (dutyratio + 50.0) \times \frac{4095}{100}$) 또한, 범위를 초과할 경우를 대비하여 Saturation 처리를 수행하여 시스템 안정성을 확보하였다.
- 0x000(0%)에서 0xFFFF(100%)까지의 12비트 해상도를 확인하였으며, I/O 보드 J4의 2번, 3번 핀을 관찰하여 정지 상태에 해당하는 50% Duty(0x800)를 기준으로 파형을 관찰하였다.

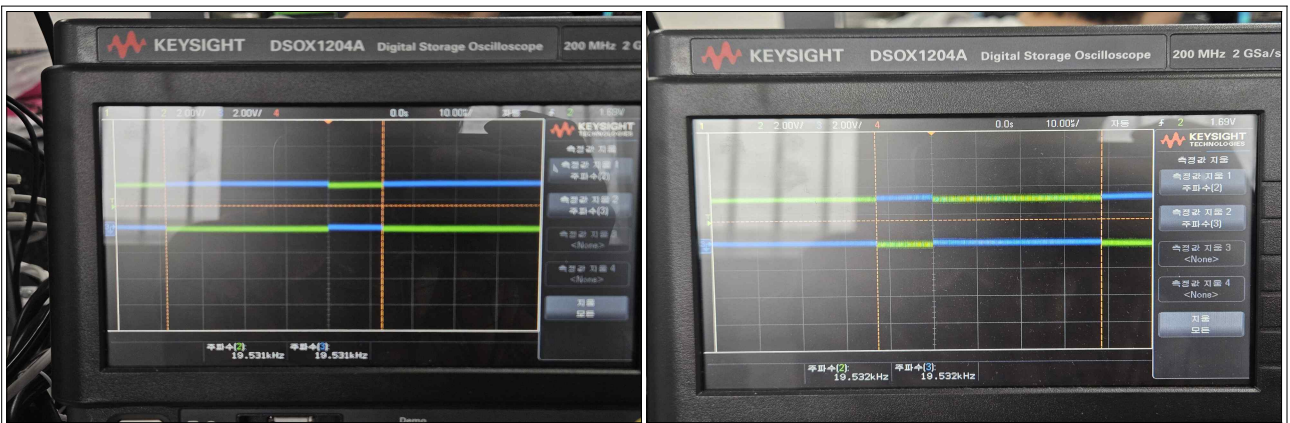


그림 1 오실로스코프로 측정한 PWM 파형

- 오실로스코프로 측정하였을 때, 소프트웨어 입력값의 변화에 따라 PWM 파형의 Duty 비가 적절하게 가변되는 것을 확인하였다. 특히 정지 상태를 기준으로 입력 부호에 따라 듀티 폭이 증감하며 모터의 속도와 방향을 제어할 수 있는 상태임을 검증하였다.

○ 결과 분석 및 결론

- 예비보고서에 작성하였듯, 소프트웨어 상에서 설정한 Duty 값이 하드웨어 레지스터를 통해 실제 전압 평균값으로 변환되는 과정을 관찰할 수 있었다.
- 작성한 PWMOut 함수가 이후 실습의 PID 제어 입력(U_{sat})으로 활용될 준비가 되었음을 확인하였다.

II

실습 2. 엔코더 신호를 이용한 위치 측정

□ 실험 과정 및 결과

○ 엔코더 파형 및 카운터 동작 확인

- J4의 4, 5번 핀에서 발생하는 A, B상 신호를 관찰하였다. 바퀴를 손으로 회전시켰을 때 회전 방향에 따라 *DC_ENC_POSCNT_R의 32비트 카운트 값이 증가 또는 감소하는 것을 UART 통신을 통해 실시간으로 확인하였다.

○ 엔코더 분해능 및 기어비 계산

- 예비보고서에 작성하였듯, 본 실험 장비의 DC 모터는 1회전 당 512회의 펄스가 생성된다. 또한, 바퀴 1회전은 DC 모터 7.5회전에 해당하므로 바퀴 1회전당 발생하는 펄스는 다음과 같다. (바퀴 1회전 당 펄스 수: $512 \times 7.5 = 3,840 \text{ counts}$)
- 이를 감안하면 1체배 기준 분해능은 $360^\circ / 3,840 \approx 0.09375^\circ / \text{pulse}$ 임을 확인할 수 있고 2체배, 4체배 설정 변경 시 카운트 값이 2배(7,680), 4배(15,360)로 정밀해짐을 확인하였다.

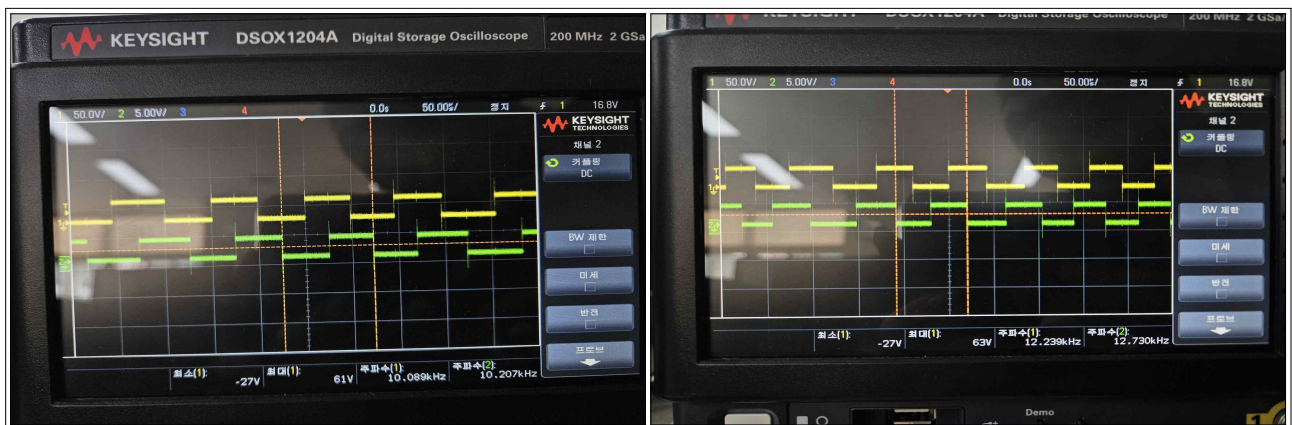


그림 2 오실로스코프로 측정한 엔코더 파형

- 오실로스코프로 엔코더 파형을 측정하였을 때, 두 상의 위상차는 90도로 동일하게 유지되지만 바퀴를 좌측(역방향)으로 회전시킨 결과와 우측(정방향)으로 회전시킨 결과가 정확하게 **반대의 위상 관계를 띄는 것**을 확인할 수 있었다.

- GetAngle 함수 구현을 통해 *DC_ENC_POSCNT_R의 카운트 값을 읽어 실제 바퀴의 회전

각도(Degree)로 환산하는 함수를 작성하였다. (환산 수식: $\angle = \text{Count} \times \frac{360.0}{3840.0}$)

○ 결과 분석 및 결론

- 기계적인 회전 운동이 전기적 펄스를 거쳐 디지털 수치로 변환되는 과정을 정량적으로 분석하였다.
- 이번 실습에서는 1체배 기준으로 진행하였기 때문에, 코드 수식에 3,840이라는 수치를 고정적으로 사용했지만 2체배, 4체배에도 대응할 수 있게 변수(Count 값)로 두어야함을 알게 되었다.
- 이 과정을 통해 도출된 GetAngle 함수는 PID 피드백 루프에서 현재 상태량(y)을 결정하는 핵심 요소가 되었다.

III

실습 3. PID 제어기 구현

□ 실험 과정 및 결과

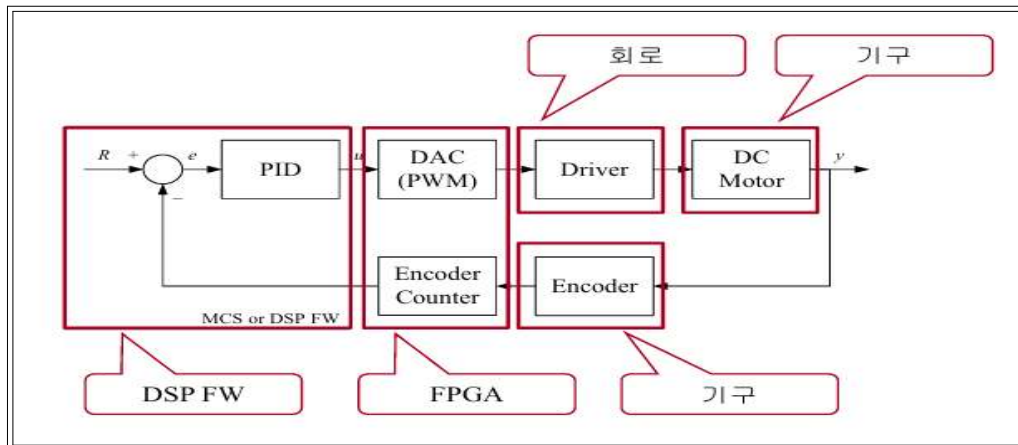


그림 3 전체 제어 시스템 구조

- 구현: DSP F/W의 Timer ISR(주기: 1kHz) 내부에 Digital PID 제어 알고리즘을 설계하였다.
- 제어 루프: GetAngle() 함수를 통해 현재 바퀴의 회전 각도를 피드백 받고, 연산된 제어 출력을 PWMOut() 함수에 전달하여 모터의 구동력을 제어하도록 구성하였다.
- 모니터링: 정확한 데이터 분석을 위해 USBMonitor를 활용하여 실시간 파형을 관측하였다.
 1. Channel 0: Reference Angle (목표 각도)
 2. Channel 1: Measured Angle (y, 측정 각도)
 3. Channel 2: Control Error (오차)
 4. Channel 3: Control Input (U_{sat}, 포화 제어 입력)

○ PID Gain 튜닝 과정

- 목표 각도(Reference)를 90도로 설정한 후, Step Response 입력을 바탕으로 PID Gain을 최적화하였다.

1. 초기 P-gain 설정 (P=0.1, I=0, D=0)

- **현상:** 시스템이 목표 각도에 도달하지 못하고 약 82도 부근에서 멈추는 정상상태 오차가 발생하였다.
- **원인 분석:** Plant 내부에 존재하는 정지 마찰력을 극복할 만큼의 충분한 제어 입력이 생성되지 않았기 때문으로 판단된다.

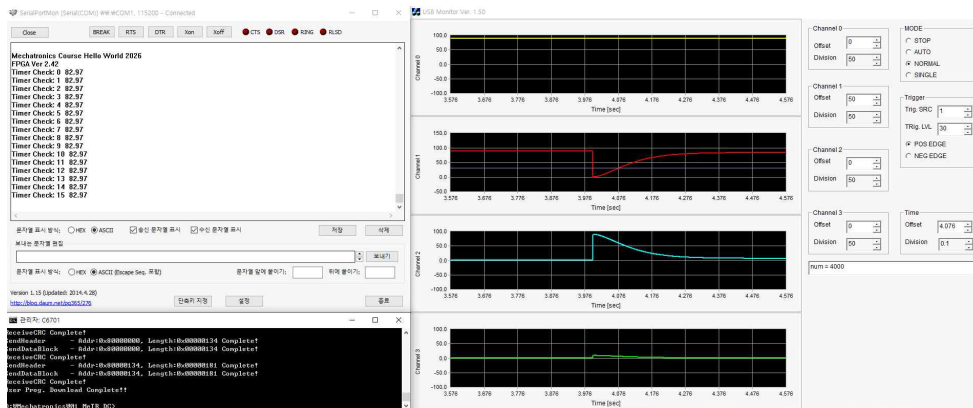


그림 4 P 제어 응답 파형 (P=0.1, I=0, D=0)

2. P-gain 증폭 및 응답성 개선 (P=1.0, I=0, D=0)

- **현상:** P-gain을 1.0으로 높인 결과, 응답 속도가 빨라지고 최종 도달 각도가 약 90도에 도달하며 오차가 거의 줄어들었다.
- **부작용:** 제어 출력이 강해진 만큼 관성에 의해 목표치를 지나치는 오버슈트가 뚜렷하게 관찰되었다.

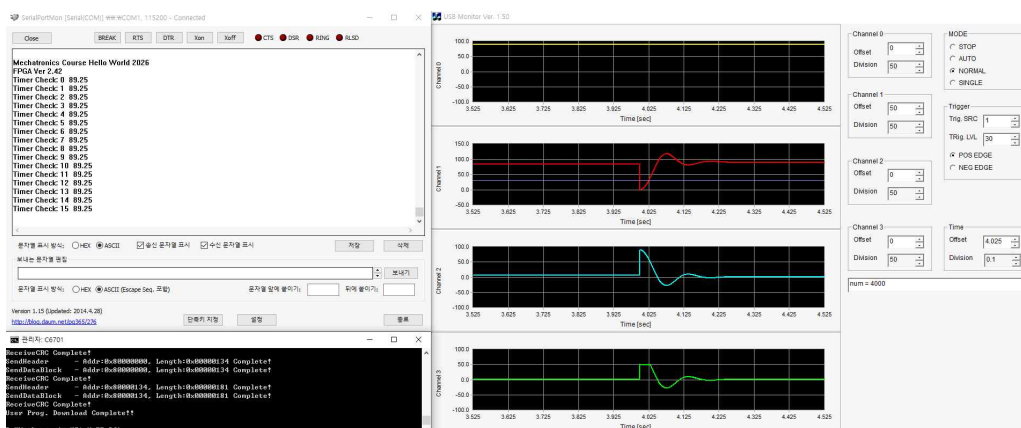


그림 5 P 제어 응답 파형 (P=1.0, I=0, D=0)

3. D-gain 추가를 통한 오버슈트 억제 (P=1.0, I=0, D=16.0)

- **현상:** Damping 효과를 주기 위해 D-gain을 16.0으로 설정하였다.
- **결과:** 빠른 응답 특성을 유지하면서도, 앞서 발생했던 오버슈트가 효과적으로 억제되어 안정적으로 목표 각도에 수렴하는 것을 확인하였다.

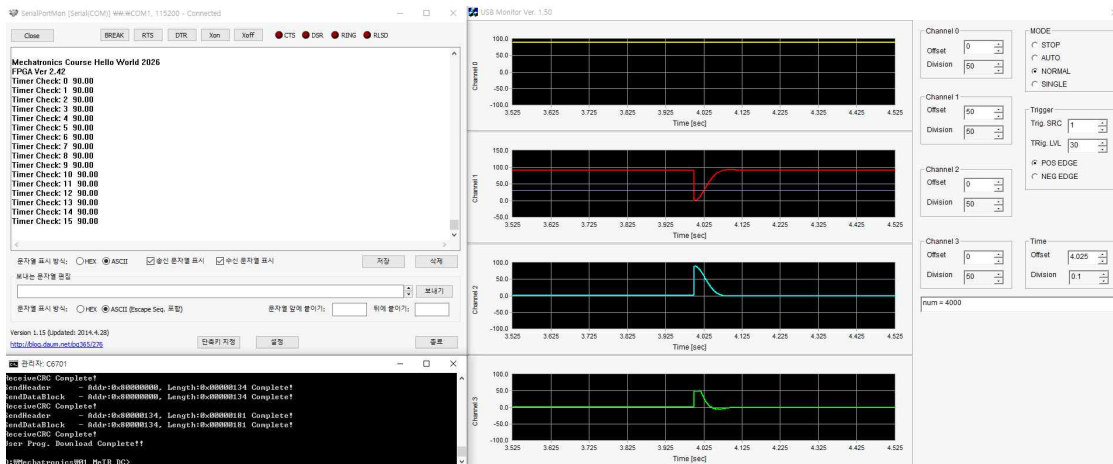


그림 6 PD 제어 최종 응답 (P=1.0, I=0, D=16.0)

○ 결과 분석 및 결론

- 예비보고서에 작성하였듯, 육안만으로 모터의 동작을 확인하지 않고 USBmonitor를 활용하여 정량적으로 분석을 수행하였다.
- 이를 통해 Reference Angle 대비 측정값(y)의 추종 성능과 제어 입력의 포화 여부를 실시간으로 확인하며 정밀한 튜닝이 가능했다.
- 결과적으로 P, I, D 게인의 물리적 역할을 이해하고, 시스템의 마찰력과 관성 특성을 고려하여 안정적인 위치 제어를 구현하였다.

IV

실습 4. 위치 궤적 트래킹

□ 실험 과정 및 결과

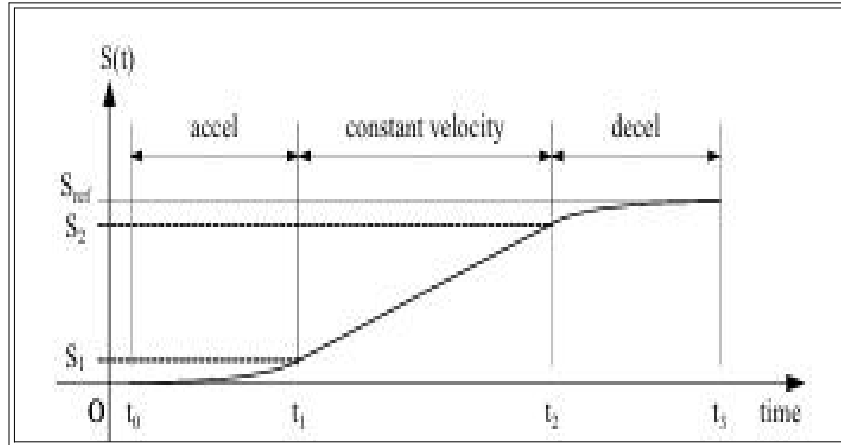


그림 7 위치 궤적 트래킹 제어

- **목적:** 급격한 목표값 변화(Step Input)로 인한 시스템의 충격을 방지하고, 시간에 따른 부드러운 위치 궤적을 추종하는 트래킹 제어 성능을 분석한다.
- **구현 방식:** 사다리꼴 속도 프로파일 알고리즘을 설계하여 목표 각도(R)를 시간에 따른 연속적인 함수로 생성하였다.
- **파라미터 설정:** 이동 거리(S_{ref}), 최대 속도(V_{max}), 가속도(a)를 변수로 설정하여 궤적의 형태를 결정하였다.
- **모니터링 (USBMonitor)**
 1. **Channel 0:** Reference Angle (생성된 목표 위치 궤적)
 2. **Channel 1:** Velocity Profile (속도 프로파일)
 3. **Channel 2:** Tracking Error (추종 오차)
 4. **Channel 3:** Control Input (U_{sat} , 포화 제어 입력)

○ Feedforward 제어가 없는 위치 트래킹

1. 일반적인 사다리꼴 프로파일

- 설정값: $a = 3000$, $V_{\max} = 500$, $S_{ref} = 360$
- 현상: 목표 이동 거리(S_{ref})가 가속에 필요한 최소 거리($V_{\max}^2/a$)보다 크므로 가속-등속-감속 구간이 모두 나타나는 사다리꼴 속도 파형이 생성되었다.
- 분석: PID 제어기만으로는 목표 궤적을 실시간으로 따라가는 데 시간 지연에 따른 추종 오차가 발생하였다. P-gain을 높여 오차를 줄일 수 있으나, 이는 시스템의 Stability를 저해하고 진동을 유발할 위험이 있음을 확인하였다.

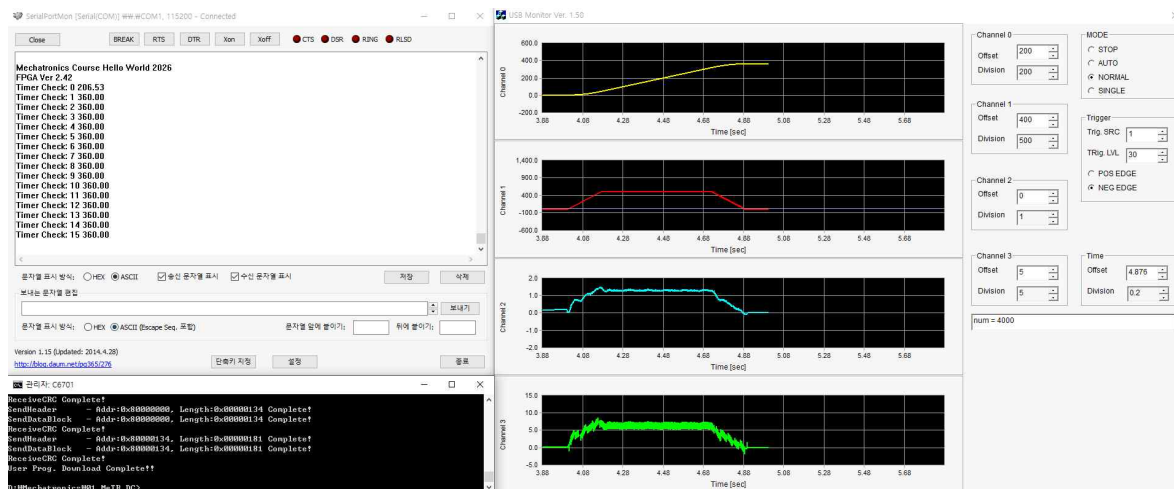


그림 8 피드포워드 미적용 시 사다리꼴 속도 프로파일 추종 결과

2. 짧은 거리 이동의 예외 상황

- 설정값: $a = 2000$, $V_{\max} = 1000$, $S_{ref} = 360$
- 현상: 가속 후 등속 속도에 도달하기 전 감속 시점에 도달하여 등속 구간 없이 삼각형 형태의 속도 프로파일이 생성되었다.
- 분석: 프로파일 생성 알고리즘이 예외 상황에서도 적절한 감속 시점을 계산하여 목표 위치에 안정적으로 도달함을 확인하였다. 다만, 피드백 제어만으로는 오차가 증가하는 한계가 있었다.

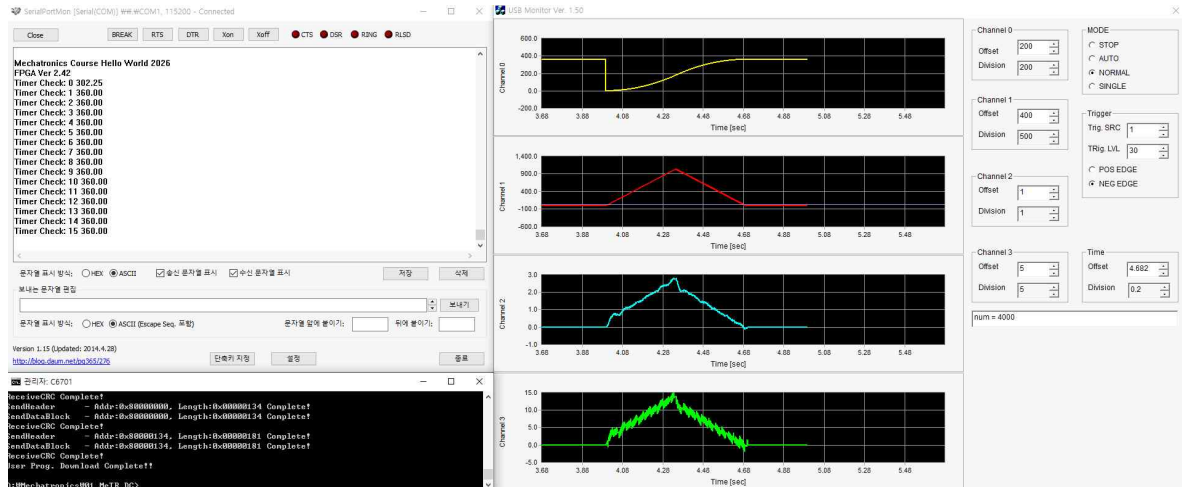


그림 9 피드포워드 미적용 시 삼각형 속도 프로파일 추종 결과

○ Feedforward 제어를 추가한 2-DOF 제어 구조

1. 일반적인 사다리꼴 프로파일

- 설정값: $a = 3000$, $V_{\max} = 500$, $S_{ref} = 360$
- 원리: 목표 궤적의 속도 정보를 이용해 필요한 제어 입력을 미리 계산하여 더해주는 Feedforward 항을 추가하였다. ($U_{total} = U_{PID} + U_{FF}$)
- 현상: FF 제어가 적용 후, P-gain을 높이지 않고도 추종 오차가 획기적으로 감소하였다.
- 분석: 피드백 제어가 오차가 발생한 후 대응하는 방식이라면 피드포워드는 목표 궤적의 움직임을 미리 예측하여 대응한다. 이로써 시스템의 안정성을 유지하면서도 정밀한 추종 성능을 동시에 달성할 수 있는 2-DOF 제어 구조의 장점을 확인하였다.

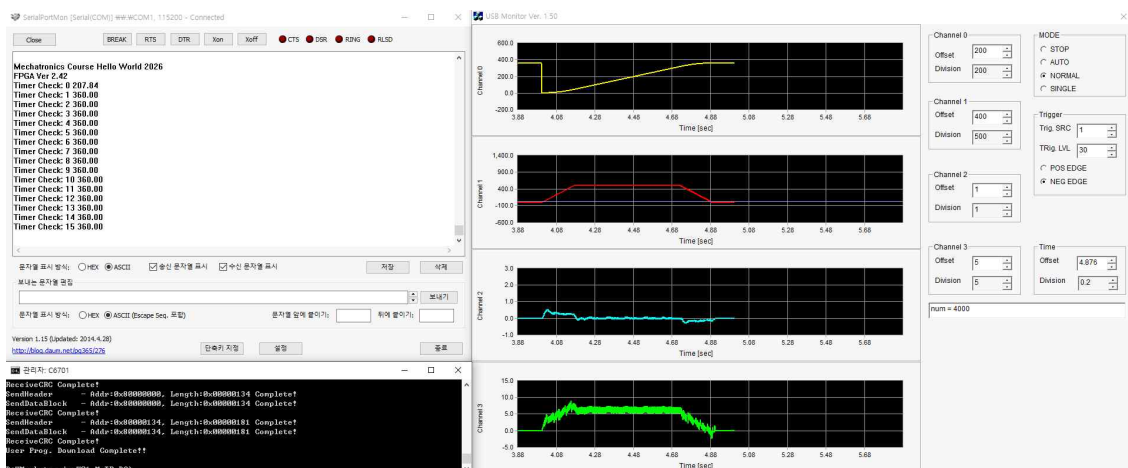


그림 10 피드포워드(2-DOF) 적용 시 사다리꼴 궤적 추종 결과

2. 짧은 거리 이동의 예외 상황

- 설정값: $a = 2000$, $V_{\max} = 1000$, $S_{ref} = 360$

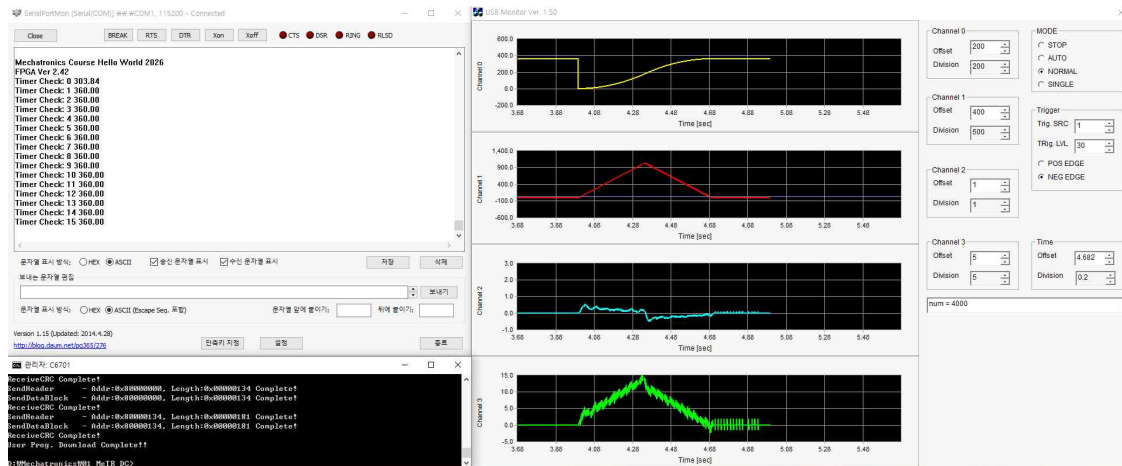


그림 11 피드포워드(2-DOF) 적용 시 삼각형 궤적 추종 결과

○ 결과 분석 및 결론

- 제어 관점: 사다리꼴 프로파일을 통한 부드러운 가감속 제어와 피드포워드 결합을 통해 모터의 위치 제어 성능을 최적화할 수 있었다.
- F/W 구현 관점: 많은 변수를 전역으로 설정할 경우 디버깅의 어려움이 발생할 수 있음을 인지하였고, 수식의 최종 결과값만 상수로 넣는 방식 대신 #define과 연산식을 그대로 코드로 구현함으로써 물리적 의미를 명확히 해야함을 알게 되었다.
- 결과적으로 실습을 통해 이론으로만 접했던 위치 궤적 생성 알고리즘과 2-DOF 제어 구조를 실제 DSP 환경에서 구현하며 유용한 제어 기법을 습득하였다.